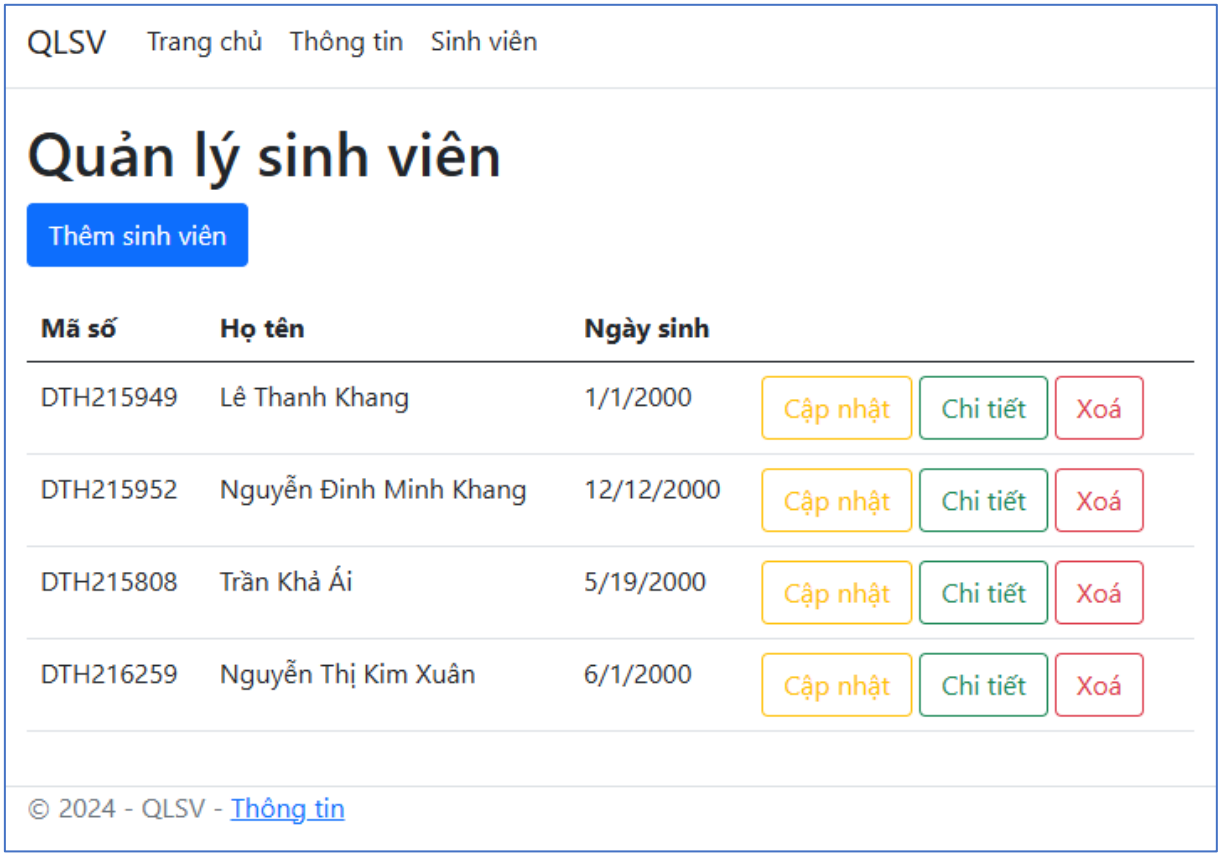
BÀI TẬP THỰC HÀNH 02



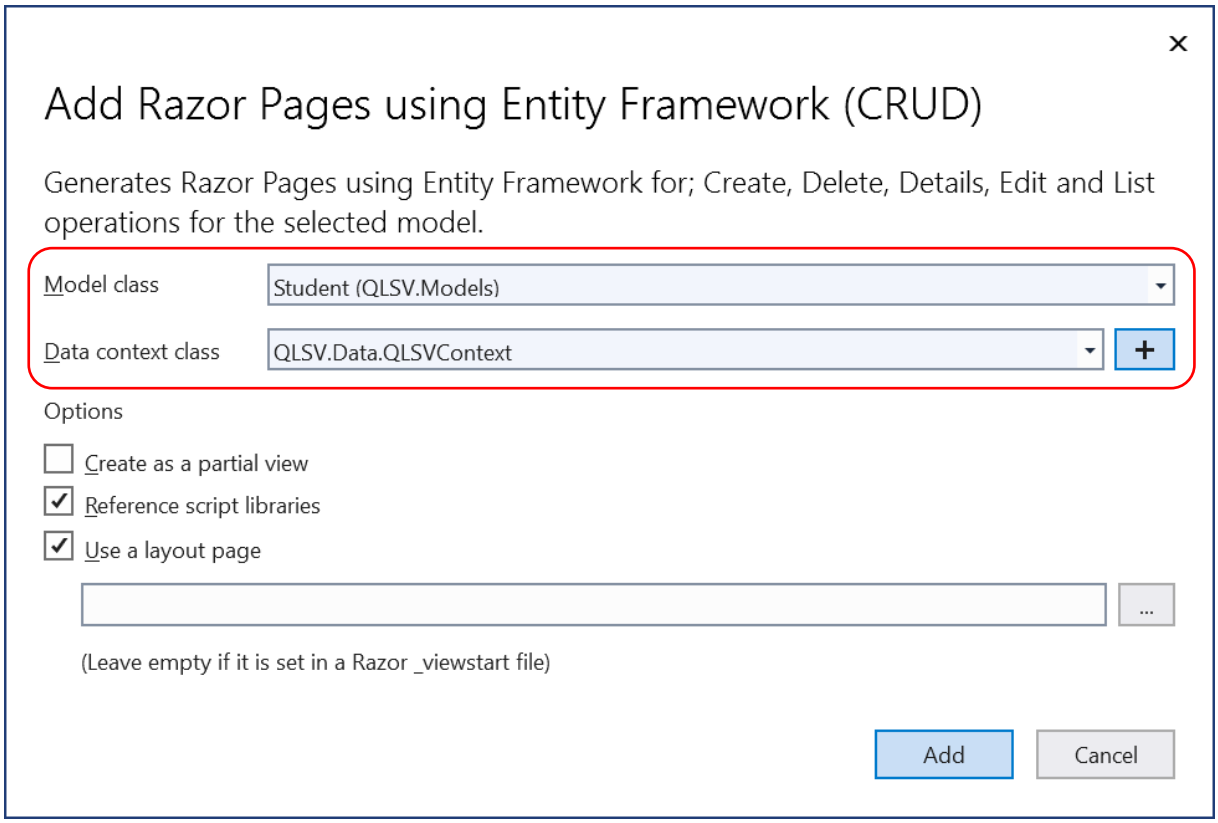
1. Tạo dự án ASP.NET Core Web App

- Tạo thư mục **Models** trong dự án. Trong thư mục **Models**, tạo mới class **Student.cs** với các thuộc tính mô tả như sau:

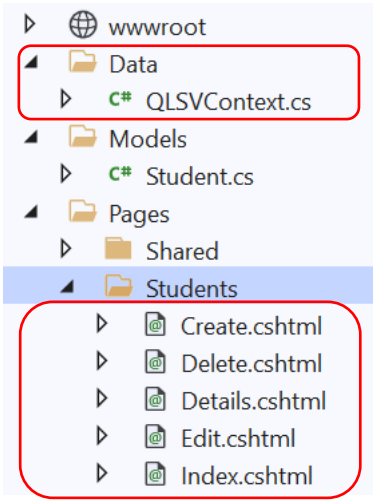
```
1   namespace qlsv.Models
2   {
        6 references
3       public class Student
4       {
            11 references
5           public int Id { get; set; }
            12 references
6           public string? Code { get; set; }
            12 references
7           public string? Name { get; set; }
            12 references
8           public DateTime Birth { get; set; }
9       }
10  }
```

- Trong thư mục **Pages**, tạo mới thư mục **Students**. Tự động phát sinh các chức năng CRUD cho lớp Student bằng cách:
  - click phải thư mục **Students**, chọn **Add**, **New Scaffolded Items**,

– chọn **Razor Pages using Entity Framework (CRUD)**, đặt các thông số như hình:



- Quan sát các file mới được tạo ra:



- Khởi tạo migration và cập nhật database từ cửa sổ **Package Manager Console**:

```
Add-Migration InitialCreate
Update-Database
```

- Chạy thử ứng dụng. Kiểm tra các chức năng Create, Detail, Edit, Delete.

2. Khởi tạo dữ liệu với SeedData

- Trong thư mục **Models**, tạo mới class **SeedData.cs** với nội dung sau:

```
using Microsoft.EntityFrameworkCore;
using QLSV.Data;
using QLSV.Models;

namespace QLSV.Models
{
    public class SeedData
    {
        public static void Initialize(IServiceProvider serviceProvider)
        {
            using (var context = new QLSVContext(
            serviceProvider.GetRequiredService<DbContextOptions<QLSVContext>>()))
            {
                if (context == null || context.Student == null)
                {
                    throw new ArgumentNullException("Null QLSVContext");
                }
                // Kiểm tra đã có dữ liệu hay chưa
                if (context.Student.Any())
                {
                    return;
                }
                // Chưa có dữ liệu, chạy data này
                context.Student.AddRange(
                    new Student
                    {
                        Code = "DTH215949",
                        Name = "Lê Thanh Khang",
                        Birth = DateTime.Parse("2000-1-1")
                    },
                    new Student
                    {
                        Code = "DTH215952",
                        Name = "Nguyễn Đinh Minh Khang",
                        Birth = DateTime.Parse("2000-12-12")
                    },
                    new Student
                    {
                        Code = "DTH215808",
                        Name = "Trần Khả Ái",
                        Birth = DateTime.Parse("2000-5-19")
                    },
                    new Student
                    {
                        Code = "DTH216259",
                        Name = "Nguyễn Thị Kim Xuân",
                        Birth = DateTime.Parse("2000-6-1")
                    }
                );
                context.SaveChanges();
            }
        }

    }
}
```

- Gọi phương thức khởi tạo dữ liệu trong **Program.cs**:

```
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.DependencyInjection;
using QLSV.Data;
using QLSV.Models;

var builder = WebApplication.CreateBuilder(args);
```

```
// Add services to the container.
builder.Services.AddRazorPages();
builder.Services.AddDbContext<QLSVContext>(options =>
    options.UseSqlServer(builder.Configuration.GetConnectionString("QLSVContext")
?? throw new InvalidOperationException("Connection string 'QLSVContext' not
found.")));

var app = builder.Build();

using (var scope = app.Services.CreateScope())
{
    var services = scope.ServiceProvider;

    SeedData.Initialize(services);
}

// Configure the HTTP request pipeline.
if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Error");
    // The default HSTS value is 30 days. You may want to change this for
production scenarios, see https://aka.ms/aspnetcore-hsts.
    app.UseHsts();
}

app.UseHttpsRedirection();
app.UseStaticFiles();

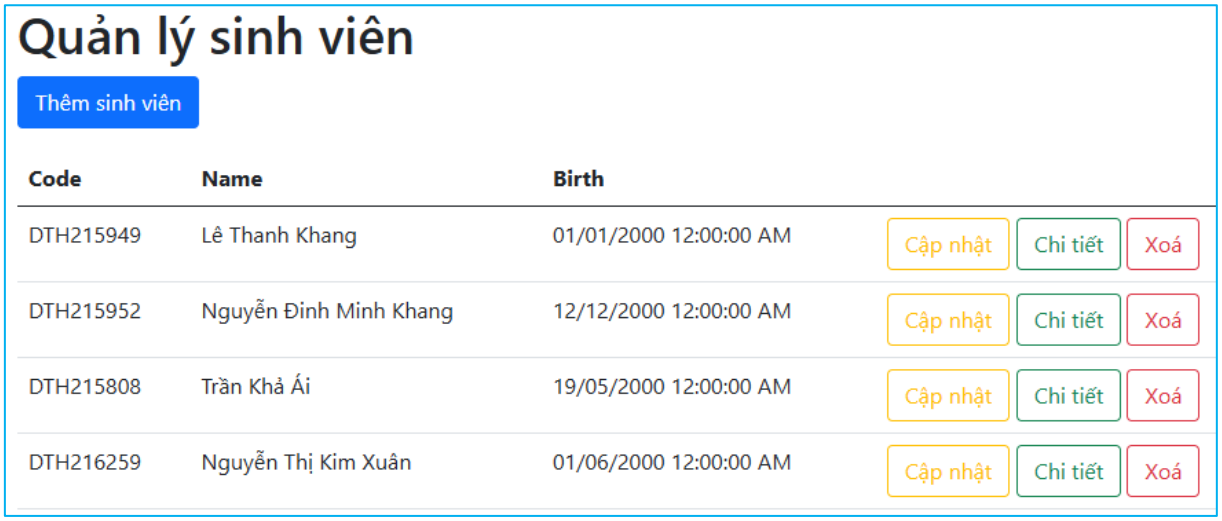
app.UseRouting();

app.UseAuthorization();

app.MapRazorPages();

app.Run();
```

- Xóa hết dữ liệu (nếu có), chạy ứng dụng để xem kết quả.



3. Ràng buộc dữ liệu

- Mở tập tin **Student.cs**, bổ sung các quy tắc ràng buộc cho các thuộc tính:

```
namespace QLSV.Models
{
    public class Student
    {
```

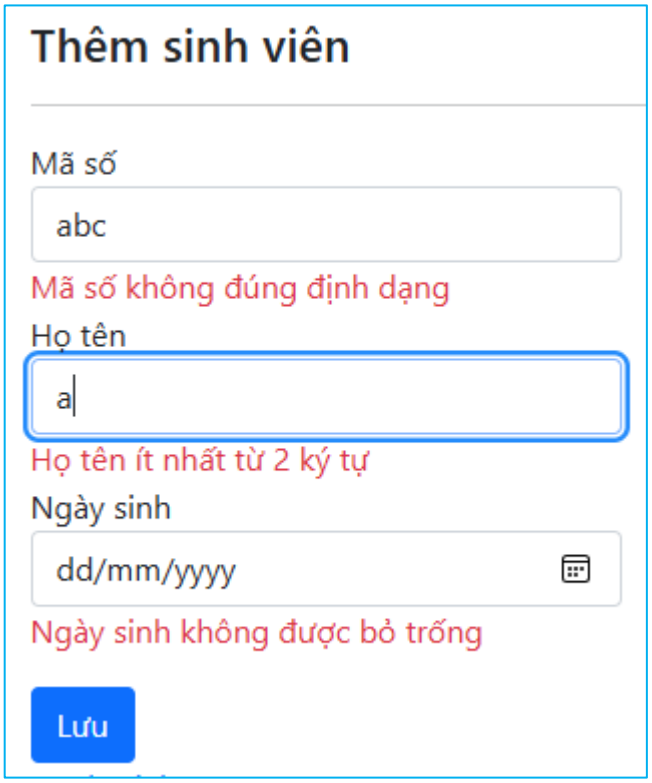
```
        public int Id { get; set; }

        [RegularExpression(@"[A-Z]{3}[0-9]{6}")]
        [Display(Name="Mã số")]
        [StringLength(9)]
        [Required]
        public string? Code { get; set; }

        [Display(Name = "Họ tên")]
        [StringLength(50, MinimumLength = 2)]
        [Required]
        public string? Name { get; set; }

        [Display(Name = "Ngày sinh")]
        [DataType(DataType.Date)]
        public DateTime Birth { get; set; }
    }
}
```

- Chạy thử ứng dụng. Kiểm tra các ràng buộc trong chức năng Create, Edit, ta thấy các thông báo lỗi nếu nhập dữ liệu không thỏa ràng buộc:



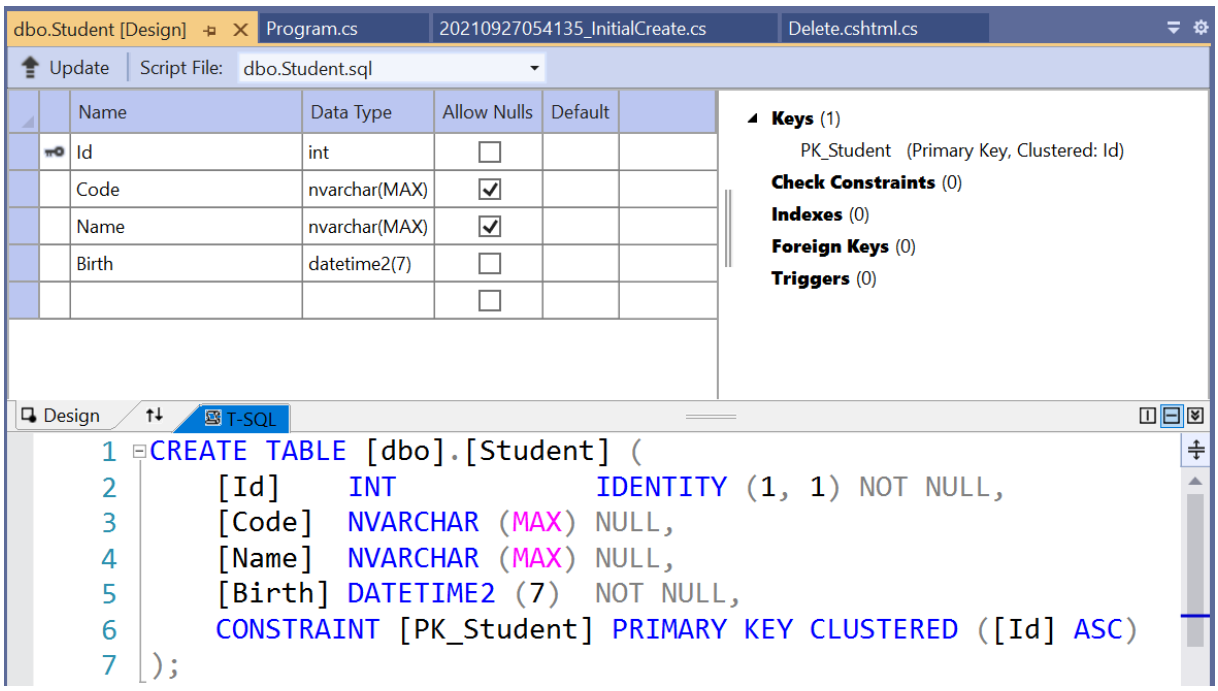
- Chạy migration và cập nhật database từ cửa sổ **Package Manager Console**:

```
Add-Migration New_DataAnnotations

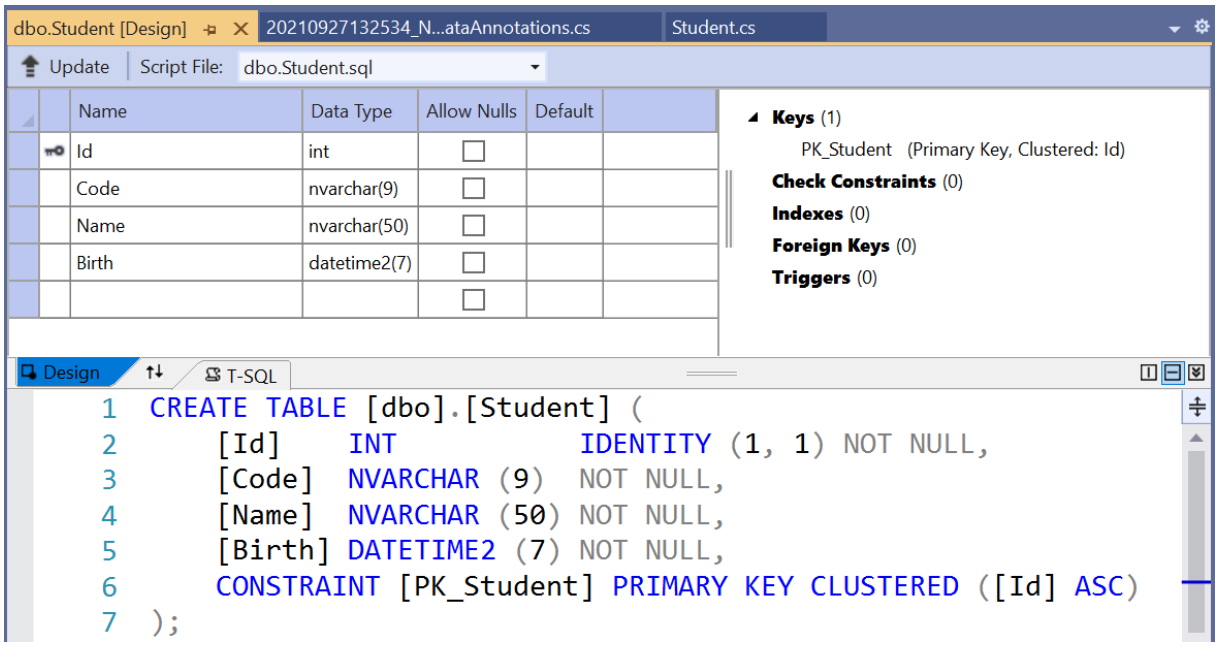
Update-Database
```

Kiểm tra lại kiểu dữ liệu các trường trong database trước và sau cập nhật:

- Database trước khi cập nhật:



- Database sau khi cập nhật:



4. Cập nhật bản thiết kế dữ liệu

- Mở tập tin **Student.cs**, bổ sung thêm thuộc tính Email (sau đó Build dự án).

```
public string Email { get; set; }
```

- Lần lượt bổ sung trường Email cho các trang Index, Create, Detail, Update, Delete. Ví dụ trang **Index.cshtml**:

```
@page
@model QLSV.Pages.Students.IndexModel

@{
    ViewData["Title"] = "Danh sách sinh viên";
```

```
}

<h1>Quản lý sinh viên</h1>

<p>
    <a class="btn btn-info" asp-page="Create">Thêm sinh viên</a>
</p>
<table class="table">
    <thead>
        <tr>
            <th>
                @Html.DisplayNameFor(model => model.Student[0].Code)
            </th>
            <th>
                @Html.DisplayNameFor(model => model.Student[0].Name)
            </th>
            <th>
                @Html.DisplayNameFor(model => model.Student[0].Birth)
            </th>
            <th>
                @Html.DisplayNameFor(model => model.Student[0].Email)
            </th>
            <th></th>
        </tr>
    </thead>
    <tbody>
@foreach (var item in Model.Student) {
        <tr>
            <td>
                @Html.DisplayFor(modelItem => item.Code)
            </td>
            <td>
                @Html.DisplayFor(modelItem => item.Name)
            </td>
            <td>
                @Html.DisplayFor(modelItem => item.Birth)
            </td>
            <td>
                @Html.DisplayFor(modelItem => item.Email)
            </td>
            <td>
                <a class="btn btn-outline-warning" asp-page="./Edit" asp-route-
id="@item.Id">Cập nhật</a>
                <a class="btn btn-outline-success" asp-page="./Details" asp-route-
id="@item.Id">Chi tiết</a>
                <a class="btn btn-outline-danger" asp-page="./Delete" asp-route-
id="@item.Id">Xóa</a>
            </td>
        </tr>
}
    </tbody>
</table>
```

- Chạy migration và cập nhật database từ cửa sổ **Package Manager Console**:

  Add-Migration AddEmail

  Update-Database

- Chạy thử ứng dụng.